

Constexpr for swap and swap related functions

I. Introduction and Motivation

The Standard Library provides a great collection of algorithms, many of which currently lack constexpr support. Even a simple constexpr usage requires reimplementing a big bunch of the Standard Library. Consider the simple example:

```
#include <array>
#include <algorithm>

int main() {
    // OK
    constexpr std::array<char, 6> a { 'H', 'e', 'l', 'l', 'o' };

    // Failures:
    // * std::find is not constexpr
    constexpr auto it = std::find(a.rbegin(), a.rend(), 'H');
}
```

All the required algorithms already were approved by LEWG in [P0202R2](#), however P0202R2 was reduced and accepted as [P0202R3](#) because of the [CWG 1581](#). Now, when CWG 1581 is fixed this paper provides wording for making with constexpr the algorithms that were not marked by P0202R3 but were in P0202R2.

Adding constexpr to algorithms that use numeric algorithms, searchers, some of the functions in char_traits and functions that rely on constexpr algorithms (like std::array comparison operators) will be covered in separate papers.

II. Impact on the Standard

This proposal is a pure library extension. It proposes changes to existing headers <utility> and <algorithm> such that the changes do not break existing code and do not degrade performance. It does not require any changes in the core language in simple cases of non assembly optimized Standard Library, and it could be implemented in standard C++.

III. Design Decisions

A. <cstring> must not have constexpr additions

B. Assumption that it is possible to implement all the proposed changes without affecting language core, especially [expr.const]

There are many Standard Library implementations nowadays, including some proprietary. It is impossible to investigate all of them to be 100% sure that no performance degradation is possible.

This proposal assumes that:

- If algorithm uses compiler intrinsics, then those intrinsics could be made constexpr by compiler vendors.
- If algorithm uses assembly optimization, then that assembly could be turned into constexpr compiler intrinsic.
- If algorithm uses external functions, then those functions could be made inline and marked constexpr or could be replaced with intrinsics.
- Modern compilers are good in code optimization, so a decently small amount of algorithms use assembly or intrinsics.

C. Analysis of existing <algorithm> implementations.

Algorithms stable_partition, inplace_merge and stable_sort allocate memory, construct variables using placement new, use unique_ptr and do other things not acceptable in constexpr expressions. Making those algorithms constexpr seems to be a hard task that would require a lot of intrinsics. Those algorithms are not marked with constexpr in this wording.

Algorithms shuffle and sample rely upon uniform_int_distribution that has no constexpr functions. Those algorithms are not marked with constexpr in this wording.

libc++ uses goto in some algorithms, this must be pretty simple to fix without affecting performance.

D. Do not mark ExecutionPolicy&& overloads with constexpr.

It seems that N4687 accidentally marks some of the ExecutionPolicy&& overloads with constexpr execution policy. This wording does not mark the ExecutionPolicy&& overloads with constexpr.

IV. Proposed wording relative to N4713

All the additions to the Standard are marked with underlined green.

A. Modifications to "Header <algorithm> synopsis" [algorithms.general]

Note for editor: All the functions in [algorithms.general] must be marked with constexpr, except functions shuffle, sample, stable_sort, stable_partition, inplace_merge, functions accepting ExecutionPolicy.

```
// 28.6.3, swap
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator2 swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 swap_ranges(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2);

template<class ForwardIterator1, class ForwardIterator2>
constexpr void iter_swap(ForwardIterator1 a, ForwardIterator2 b);

// 28.6.10, reverse
template<class BidirectionalIterator>
constexpr void reverse(BidirectionalIterator first, BidirectionalIterator last);

template<class ExecutionPolicy, class BidirectionalIterator>
void reverse(ExecutionPolicy&& exec, // see 28.4.5
    BidirectionalIterator first, BidirectionalIterator last);

template<class BidirectionalIterator, class OutputIterator>
constexpr OutputIterator reverse_copy(BidirectionalIterator first, BidirectionalIterator last, OutputIterator result);

template<class ExecutionPolicy, class BidirectionalIterator, class ForwardIterator>
ForwardIterator reverse_copy(ExecutionPolicy&& exec, // see 28.4.5
    BidirectionalIterator first,
    BidirectionalIterator last,
    ForwardIterator result);

// 28.6.11, rotate
template<class ForwardIterator>
constexpr ForwardIterator rotate(ForwardIterator first, ForwardIterator middle, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator rotate(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first,
    ForwardIterator middle,
    ForwardIterator last);

template<class ForwardIterator, class OutputIterator>
constexpr OutputIterator rotate_copy(ForwardIterator first, ForwardIterator middle, ForwardIterator last, OutputIterator result);

template<class ExecutionPolicy, class ForwardIterator, class ForwardIterator>
ForwardIterator rotate_copy(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator middle,
    ForwardIterator last, ForwardIterator result);

// 28.7.4, partitions
template <class InputIterator, class Predicate>
constexpr bool is_partitioned(InputIterator first, InputIterator last, Predicate pred);

template <class ExecutionPolicy, class ForwardIterator, class Predicate>
bool is_partitioned(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last, Predicate pred);

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator partition(ForwardIterator first, ForwardIterator last, Predicate pred);

template<class ExecutionPolicy, class ForwardIterator, class Predicate>
ForwardIterator partition(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first,
    ForwardIterator last,
    Predicate pred);

template<class BidirectionalIterator, class Predicate>
BidirectionalIterator stable_partition(BidirectionalIterator first, BidirectionalIterator last, Predicate pred);

template<class ExecutionPolicy, class BidirectionalIterator, class Predicate>
BidirectionalIterator stable_partition(ExecutionPolicy&& exec, // see 28.4.5
    BidirectionalIterator first,
    BidirectionalIterator last,
    Predicate pred);

template <class InputIterator, class OutputIterator1, class OutputIterator2, class Predicate>
constexpr pair<OutputIterator1, OutputIterator2>
    partition_copy(InputIterator first, InputIterator last, OutputIterator1 out_true, OutputIterator2 out_false, Predicate pred);

template <class ExecutionPolicy, class ForwardIterator, class ForwardIterator1, class ForwardIterator2, class Predicate>
pair<ForwardIterator1, ForwardIterator2>
    partition_copy(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    ForwardIterator1 out_true, ForwardIterator2 out_false,
    Predicate pred);

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator partition_point(ForwardIterator first, ForwardIterator last, Predicate pred);

// 28.7, sorting and related operations
// 28.7.1, sorting
template<class RandomAccessIterator>
```

```

constexpr void sort(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void sort(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
void sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last,
    Compare comp);

template<class RandomAccessIterator>
void stable_sort(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
void stable_sort(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
void stable_sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void stable_sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last,
    Compare comp);

template<class RandomAccessIterator>
constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator middle, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator middle, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
void partial_sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first,
    RandomAccessIterator middle,
    RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void partial_sort(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first,
    RandomAccessIterator middle,
    RandomAccessIterator last, Compare comp);

template<class InputIterator, class RandomAccessIterator>
constexpr RandomAccessIterator partial_sort_copy(InputIterator first, InputIterator last, RandomAccessIterator result_first, RandomAccessIterator result_last);

template<class InputIterator, class RandomAccessIterator, class Compare>
constexpr RandomAccessIterator partial_sort_copy(InputIterator first, InputIterator last, RandomAccessIterator result_first, RandomAccessIterator result_last, Compare comp);

template<class ExecutionPolicy, class ForwardIterator, class RandomAccessIterator>
RandomAccessIterator partial_sort_copy(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last);

template<class ExecutionPolicy, class ForwardIterator, class RandomAccessIterator, class Compare>
RandomAccessIterator partial_sort_copy(
    ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last,
    Compare comp);

template<class ForwardIterator>
constexpr bool is_sorted(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
constexpr bool is_sorted(ForwardIterator first, ForwardIterator last, Compare comp);

template<class ExecutionPolicy, class ForwardIterator>
bool is_sorted(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator, class Compare>
bool is_sorted(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Compare comp);

template<class ForwardIterator>
constexpr ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
constexpr ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last, Compare comp);

template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator is_sorted_until(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator, class Compare>
ForwardIterator is_sorted_until(ExecutionPolicy&& exec, // see 28.4.5
    ForwardIterator first, ForwardIterator last,
    Compare comp);

```

```

// 28.7.2, Nth element
template<class RandomAccessIterator>
constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
void nth_element(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator nth,
    RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void nth_element(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator nth,
    RandomAccessIterator last, Compare comp);

// 28.7.7, heap operations
template<class RandomAccessIterator>
constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr void pop_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void pop_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr void sort_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void sort_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr bool is_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr bool is_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
bool is_heap(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
bool is_heap(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class ExecutionPolicy, class RandomAccessIterator>
RandomAccessIterator is_heap_until(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
RandomAccessIterator is_heap_until(ExecutionPolicy&& exec, // see 28.4.5
    RandomAccessIterator first, RandomAccessIterator last,
    Compare comp);

// 28.7.11, permutations
template<class BidirectionalIterator>
constexpr bool next_permutation(BidirectionalIterator first, BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr bool next_permutation(BidirectionalIterator first, BidirectionalIterator last, Compare comp);

template<class BidirectionalIterator>
constexpr bool prev_permutation(BidirectionalIterator first, BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr bool prev_permutation(BidirectionalIterator first, BidirectionalIterator last, Compare comp);
}

```

B. Modifications to remaining parts of "28 Algorithms library" [algorithms] (all sections except "Header <algorithm> synopsis" and "28.8 C library algorithms" [alg.c.library])

Note for editor: All the functions marked with constexpr in previous paragraph of this document must be accordingly marked with constexpr in detailed algorithm description. For shortness only modifications to "28.6.3 Swap" [alg.swap] are shown in this paper.

28.6.3 Swap [alg.swap]

```
template <class ForwardIterator1, class ForwardIterator2>
    ForwardIterator2
        constexpr swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1,
                               ForwardIterator2 first2);

template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 swap_ranges(ExecutionPolicy&& exec, ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2);

Requires: The two ranges [first1, last1) and [first2, first2 + (last1 - first1)) shall not overlap. *(first1 + n) shall be swappable with (;
Effects: For each non-negative integer n < (last1 - first1) performs: swap(*(first1 + n), *(first2 + n)).
Returns: first2 + (last1 - first1).
Complexity: Exactly last1 - first1 swaps.

template <class ForwardIterator1, class ForwardIterator2>
    constexpr void iter_swap(ForwardIterator1 a, ForwardIterator2 b);

Requires: a and b shall be dereferenceable. *a shall be swappable with (20.5.3.2) *b.
Effects: As if by swap(*a, *b).
```

C. Modifications to "23.2 Utility components" [utility]

```
// 23.2.3, swap:
template<class T>
constexpr void swap(T& a, T& b) noexcept(see below );

template <class T, size_t N>
constexpr void swap(T (&a)[N], T (&b)[N]) noexcept(noexcept(swap(*a, *b)));

// 23.2.4, exchange:
template <class T, class U=T>
constexpr T exchange(T& obj, U&& new_val);
```

D. Modifications to "23.2.3 swap" [utility.swap]

```
template<class T> constexpr void swap(T& a, T& b) noexcept(see below );

Remark: This function shall not participate in overload resolution unless is_move_constructible_v<T>
is true and is_move_assignable_v<T> is true. The expression inside noexcept is equivalent to:
    is_nothrow_move_constructible_v<T> && is_nothrow_move_assignable_v<T>

Requires: Type T shall be MoveConstructible (Table 20) and MoveAssignable (Table 22).
Effects: Exchanges values stored in two locations.

template<class T, size_t N>
constexpr void swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable_v<T>);

Remarks: This function shall not participate in overload resolution unless is_swappable_v<T> is true.

Requires: a[i] shall be swappable with (20.5.3.2) b[i] for all i in the range [0,N).
Effects: As if by swap_ranges(a, a + N, b).
```

E. Modifications for P0858 [iterator.requirements.general]

Note for editor: Apply this change if P0858 is accepted

Iterators are called *constexpr iterators* if all operations provided to satisfy iterator category operations are constexpr functions, except for

- ~~swap~~
- a pseudo-destructor call (8.2.4 [expr.pseudo]), and
- the construction of an iterator with a singular value.

F. Feature-testing macro

For the purposes of SG10, we recommend the feature-testing macro name `__cpp_lib_constexpr_swap_algorithms`.

V. Revision History

Revision 0:

- Initial proposal
- Jacksonville voting on P0202R0:

Approve of adding constexpr to the algorithms?
SF F N A SA
9 4 3 0 0

Approve of adding constexpr to algorithms that need some compiler intrinsics, pending some compiler vendors approving?
SF F N A SA
2 8 5 1 0

VI. References

[N4713] Working Draft, Standard for Programming Language C++. Available online at www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4713.pdf

[rhalbersma] Proof of concept for some functions. Available online at <https://bitbucket.org/rhalbersma/xstd/src/42553df6107623c71163f104b6f3cc550c245b4b/include/xstd/algorithm.hpp> and <https://bitbucket.org/rhalbersma/xstd/src/42553df6107623c71163f104b6f3cc550c245b4b/include/xstd/utility.hpp>

[Boost.Algorithm] Constexpr modifiers for Boost Algorithm library. Available online at <https://github.com/boostorg/algorithm/pull/13>

[Discussion] A call to discuss asm in constexpr and constexpr <algorithm>. Available online at <https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/9sTJWsOpptE>

[P0202R0] Add Constexpr Modifiers to Functions in <algorithm> and <cstring> Headers. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0202r0.html>